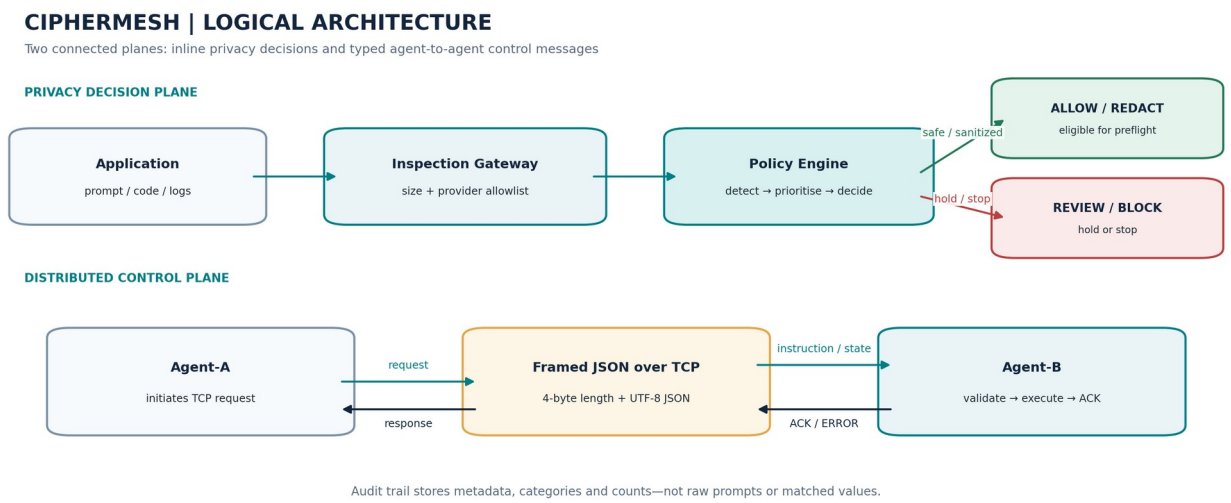
ENGINEERING PROJECT REPORT

CIPHERMESH

Multi-Cloud AI Agent

Distributed Agent Messaging & Privacy Gateway Implementation



Logical view of the privacy decision plane and distributed control plane.

Prepared by	Kumar Waibhav Akshat
Registration ID	RA2212704010027
Programme / Section	Integrated M.Tech, CSE (Data Science) / AG1
Version / Date	0.2.0 / 12 October 2026

SUBMISSION PACKAGE • IMPLEMENTATION + VERIFICATION + RUNBOOK

Executive Summary

CipherMesh is a local-first prototype of a privacy gateway designed to reduce accidental disclosure of credentials, personally identifiable information and internal infrastructure details in LLM prompts. This submission also completes the assessed distributed-agent workflow: Agent-A serializes active policy parameters and transfers them to Agent-B through a real TCP socket; Agent-B validates and applies only approved fields, then returns a correlated acknowledgement.

The implementation uses a client-server baseline, typed Pydantic envelopes, UTF-8 JSON, and a four-byte unsigned big-endian length prefix. This makes message boundaries explicit on TCP, which is a stream protocol. Bounded frame sizes, exact reads, socket timeouts, bounded worker threads, response-correlation checks and replay-ID checks provide a useful reliability baseline.

Agent-B responds proactively to `TRIGGER_EMERGENCY_LOCKDOWN`, `UPDATE_REDACTION_PATTERNS`, and `REQUEST_TELEMETRY_DUMP`. It also accepts a `STATE_SYNC` message and remote `INSPECT` requests. Audit logs record decisions and metadata only; matched sensitive values and raw prompt text are not stored.

VERIFICATION SNAPSHOT 22 automated tests passed; a live loopback TCP run verified state sync, inspection, custom pattern update, telemetry and lockdown. These results validate this baseline, not production security or benchmark performance.

Deliverables in this submission

- Runnable Python package and CLI for the Agent-A / Agent-B socket demonstration.
- Typed message schema and length-prefixed JSON transport implementation.
- Autonomous policy-state synchronization and proactive command handlers.
- Automated test suite including a real loopback TCP integration test.
- Professional implementation report, architecture diagrams, runbook and security limitations.

Reading guide

Sections 1–4 establish the goal, agent model and policy; Sections 5–8 explain transport, serialization, synchronization and proactive messages; Sections 9–11 are the implementation walkthrough and runbook; Sections 12–17 address reliability, security, evaluation, next steps and assignment mapping.

Document Map

Use this map to navigate the report. Headings are intentionally descriptive so the implementation can be reviewed independently of a live demonstration.

Section	Contents
01	Mission, scope and success criteria
02	Domain definition, stakeholders and PEAS
03	System architecture and trust boundaries
04	Condition-action policy and risk priority
05	Network topology and design rationale
06	Serialization and wire protocol
07	State synchronization and validation
08	Proactive remote-message handling
09	Codebase walkthrough
10	Setup and dual-terminal execution
11	Demonstration cases and expected outcomes
12	Stability and failure handling
13	Security and privacy analysis
14	Verification and evaluation plan
15	Limitations and next milestones
16	Assignment deliverable mapping
17	Conclusion and references
A–C	CLI quick reference, result semantics, repository inventory

1. Mission, Scope and Success Criteria

1.1 Mission

CipherMesh places a provider-neutral inspection boundary between an application and an LLM destination. It is intended to detect possible credential, PII and infrastructure disclosure before an application transmits content beyond its trust boundary. Its distributed control path lets a trusted peer submit a policy snapshot or request a controlled action without accepting arbitrary remote code.

1.2 Scope included in this build

- Local pattern-based inspection with ALLOW / REDACT / REVIEW / BLOCK decisions.
- Provider allow-list check and a preflight function that prepares text only for ALLOW or sanitized REDACT.
- Agent-A CLI client and Agent-B TCP listener.

- Typed Pydantic message models, JSON serialization and a four-byte length-prefixed frame.
- Validated and revisioned policy-state synchronization.
- Emergency lockdown, custom-redaction-pattern updates and metadata-only telemetry commands.
- ACK/ERROR responses, correlation validation, in-memory replay-ID window, bounded workers and timeouts.
- SQLite metadata audit, automated tests and optional FastAPI interface.

1.3 Explicit non-goals

This version does not invoke AWS Bedrock, Azure AI, Google Vertex AI or Ollama. It does not include mutual TLS, a human-review queue/UI, signed policy bundles, multi-host failover or a benchmark corpus. “Multi-cloud” describes the target integration domain and provider allow-list; it does not imply live provider adapters have been completed.

1.4 Success criterion

The networking requirement is complete when Agent-A sends `STATE_SYNC` over TCP, Agent-B deserializes and validates it, applies permitted parameters, keeps receiver-local fields unchanged, writes a metadata audit event and returns a correlated ACK. Proactive instructions must dispatch to a known handler and return an explicit result.

2. Domain Definition, Stakeholders and PEAS

2.1 Problem frame

Developer prompts can contain code fragments, configuration files, stack traces, URLs, logs and environment output. These may inadvertently expose credentials, personal information or internal infrastructure details. OWASP identifies Sensitive Information Disclosure and Excessive Agency among the risks relevant to LLM applications [1]. CipherMesh addresses a narrow defensive slice: deterministic preflight checks and bounded remote control messaging.

The gateway does not infer every user intent. A pattern match is evidence for a decision, not proof that a request is safe. A production system requires broader recognizers, labelled evaluation data and a policy for uncertainty.

2.2 Stakeholders

Stakeholder	Need	CipherMesh contribution
Developer	Avoid copying secrets into a model request	Preflight findings, sanitization, review and block decisions
Application owner	Consistent controls across destinations	Provider-neutral `prepare_request` plus allow-list
Security / privacy	Observable, auditable policy	Metadata-only SQLite events and aggregate counters
Platform / operations	Predictable remote configuration	Typed messages, policy revision and ACK/ERROR semantics
Reviewer / instructor	Reproducible demonstration	Dual-terminal runbook and TCP integration test

2.3 PEAS profile

PEAS element	CipherMesh mapping
Performance measures	Precision/recall on labelled data, false-block rate, latency, transport success, correct action execution and privacy-safe logs

PEAS element	CipherMesh mapping
Environment	Developer tools, prompt/response text, provider metadata, TCP endpoints, remote agents and policy state
Actuators	Allow, redact, require review, block, apply state revision, activate lockdown, update patterns, return telemetry and write audit metadata
Sensors	Text, provider identifier, sender/recipient, message type, frame length, schema result, detector findings and confidence

2.4 Task-environment characteristics

- Partially observable: only supplied text and metadata are available.
- Uncertain / stochastic: language ambiguity and unseen secret formats make detection imperfect.
- Sequential: prior policy revisions, lock state and message IDs influence subsequent behavior.
- Dynamic: requests arrive continuously and policy settings may change.
- Mixed state/action space: text and latency are continuous; findings and actions are discrete.
- Multi-agent and partly known: applications and agents exchange structured messages, but new intent and credential formats may be unknown.

3. System Architecture and Trust Boundaries

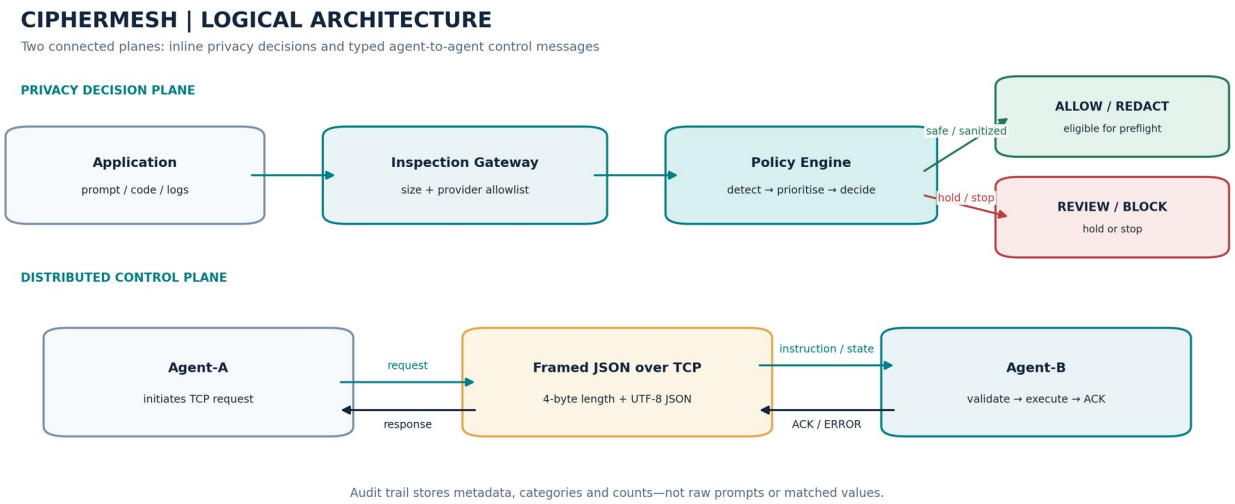


Figure 1. Logical architecture of the privacy decision plane and distributed control plane.

3.1 Privacy decision plane

The inspection path validates UTF-8 byte size, detects sensitive patterns, evaluates policy priority and returns an `InspectionResult`. The preflight layer marks only ALLOW or sanitized REDACT as forwardable. REVIEW and BLOCK return no provider content. No provider call occurs in this package; a future adapter must enforce this result at the point of transmission.

3.2 Distributed control plane

Agent-A is the initiating client and Agent-B is the listener. One request is sent on a TCP connection and one response is read. Agent-B dispatches the envelope to a narrow handler, executes the allowed operation, then returns ACK or

ERROR. A bounded thread-per-client design handles concurrent clients, while a re-entrant lock protects agent state and inspection counters.

3.3 Why client-server rather than P2P

The supplied reference proposes dual-role peer-to-peer deployment. This implementation intentionally completes the smaller client-server slice: one receiver and one sender. That makes ownership of the listener, state application and acknowledgement paths reproducible. It does not claim decentralization or elimination of single points of failure. True P2P would require listeners on both nodes, mutual identity, conflict resolution and leaderless policy-revision rules.

3.4 Endpoint conventions

Endpoint	Purpose	Status
127.0.0.1:9052	Agent-B state-sync and command listener	Implemented default
127.0.0.1:9051	Potential application-side ingress	Reserved; not implemented
127.0.0.1:8000	Optional FastAPI service	Optional

Loopback suits the local classroom demo. Private-network exposure requires encryption, firewalling and agent identity controls before the service can be considered safe.

4. Condition-Action Policy and Risk Priority

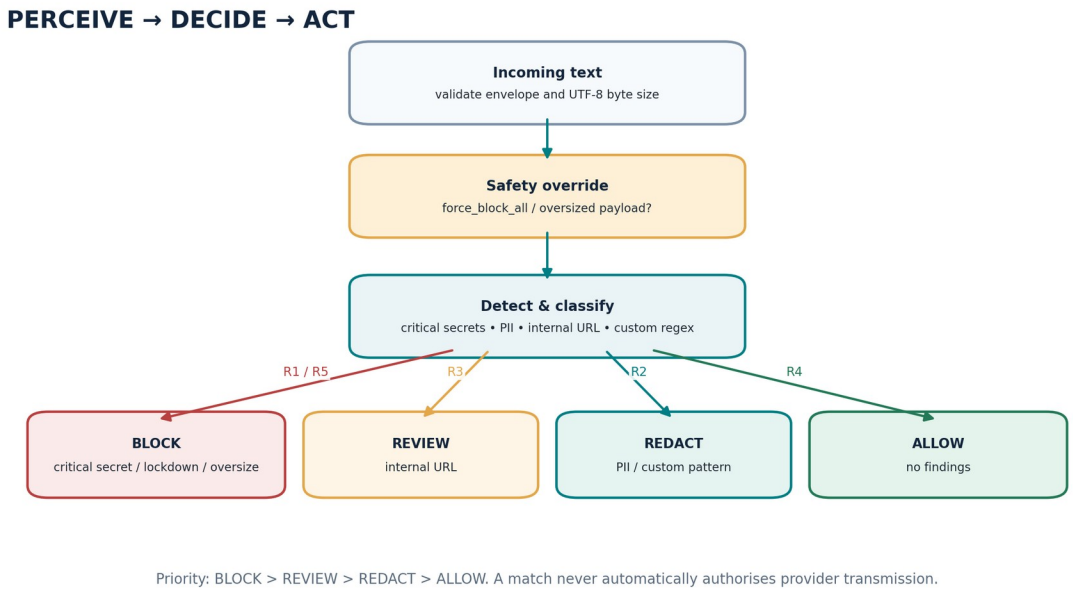


Figure 2. Perceive–decide–act flow and decision priority.

Rule	Trigger	Decision	Enforcement
R1	AWS-like access key, private-key header, Azure SAS signature or generic token	BLOCK	No provider content returned
R2	Email, phone, IPv4 or custom	REDACT	Replace matched span; only

Rule	Trigger	Decision	Enforcement
	pattern without higher priority finding		sanitized text eligible
R3	Internal/corporate URL without critical secret	REVIEW	No provider content; human review required
R4	No policy-triggering finding	ALLOW	Original text eligible through a future adapter
R5	UTF-8 content exceeds configured byte limit	BLOCK	Stop before detection
Emergency	`force_block_all` enabled	BLOCK	All content blocked until local reset flow is added

4.1 Data minimization

A finding contains category, offsets, confidence and source, but not the matched value. Redaction applies findings in descending offset order to preserve earlier positions. Audit events record categories, finding count, decision, reason and latency, not raw prompt text. Sanitized text is returned only in the immediate inspection response and is not written to SQLite.

4.2 Detection limits

The baseline uses regular expressions and catches only the configured patterns. It may miss novel, encoded, split or obfuscated secrets and may overmatch phone-like numbers. Microsoft Presidio is a possible future detector/anonymizer integration, but it is not active in the current execution path [4].

5. Network Topology and Design Rationale

5.1 Raw TCP sockets

TCP sockets make message boundaries, frame validation, receiver behavior and failure paths explicit. HTTP/JSON-RPC could be more appropriate for service orchestration and management in a larger deployment, but raw TCP directly satisfies the local-port transfer exercise.

5.2 Transport contract

Property	Baseline value / behavior
Transport	IPv4 TCP stream (`SOCK_STREAM`)
Default receiver	127.0.0.1:9052
Message pattern	One request and one response per TCP connection
Frame	4-byte unsigned big-endian length + UTF-8 JSON bytes
Maximum frame	65,536 bytes by default; server configuration capped at 1 MiB
Timeout	5-second client / server socket timeout
Concurrency	Up to 32 handler threads by default; excess clients rejected
Protocol response	Typed ACK / ERROR correlated to request ID

Python documents that a single `recv()` can return fewer bytes than requested and that `sendall()` continues sending until the bytes are sent or an error occurs [2]. CipherMesh therefore uses exact-read loops and `sendall()` instead of relying on one `recv()` call per message.

5.3 Connection lifecycle

1. Agent-B binds and listens on the configured endpoint.
2. The server accepts a connection and reserves a worker slot.
3. The handler reads the four-byte frame header and rejects zero-length or over-limit frames.
4. It reads the precise body length and validates the JSON as `WireMessage`.
5. If configured, a shared token is compared using a constant-time comparison.
6. The agent checks recipient and replay ID, dispatches a typed handler and emits a correlated ACK/ERROR.
7. Socket closes and worker capacity is released even on failure.

6. Serialization and Wire Protocol

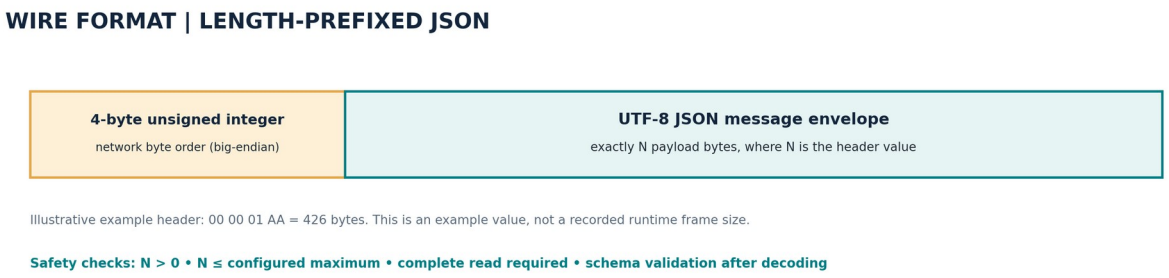


Figure 3. Length-prefixed frame layout. The 426-byte example is illustrative, not a measured runtime size.

6.1 Typed envelope

Field	Purpose
`schema_version`	Compatibility boundary; current value `1`
`message_id`	Unique request identifier and replay-window key
`message_type`	Enum such as PING, STATE_SYNC, INSPECT, COMMAND_INSTRUCTION
`sender` / `recipient`	Application-level identities
`timestamp`	UTC-capable timestamp for traceability
`correlation_id`	Response references the original request ID
`auth_token`	Optional shared token for local demo; not encryption
`payload`	Typed-by-handler JSON object, based on message type

Pydantic parses incoming data into typed models and validates declared constraints [3]. Unknown top-level envelope fields are rejected. Each message handler then validates the shape and constraints of its own payload.

6.2 Framing details

The bytes `00 00 01 AA` encode a frame length of 426. This is an illustration carried over from the supplied reference; actual byte lengths depend on the serialized JSON. The receiver reads the actual header, checks the configured limit, and only then reads the body.

6.3 Why JSON

JSON is readable in terminal screenshots and accessible to heterogeneous clients. A small frame ceiling constrains unexpectedly large messages. Binary serialization might reduce bandwidth but adds complexity not needed for this baseline.

7. State Synchronization and Validation

Agent-A sends a `STATE_SYNC` envelope with a `state` object. Agent-B only considers explicitly permitted fields: policy version/revision, provider allow-list, byte limit, review/redaction/block flags and custom pattern list. Incoming metrics are informational and never copied into Agent-B counters. Receiver identity, sequence number and emergency-lock state remain locally controlled.

7.1 Revision rules

- Higher policy revision: validate fields and apply atomically.
- Lower revision: reject as stale.
- Same revision with identical policy fields: acknowledge `state_unchanged` (idempotent repeat).
- Same revision with different policy fields: reject.
- Message IDs are kept in a bounded in-memory replay window of 4,096 IDs. This cache resets on process restart.

7.2 State-sync payload example

```
{
  "schema_version": "1",
  "message_type": "STATE_SYNC",
  "sender": "AGENT_A_NODE_01",
  "recipient": "AGENT_B_NODE_01",
  "payload": {
    "state": {
      "policy_version": "1.1",
      "policy_revision": 2,
      "provider_allowlist": ["local", "aws-bedrock", "azure-ai", "gcp-vertex"],
      "max_prompt_bytes": 65536,
      "require_review_for_internal_urls": true,
      "redact_email_phone_ip": true,
      "block_critical_secrets": true,
      "custom_redaction_patterns": []
    }
  }
}
```

The running client generates the message ID and timestamp and can attach an auth token. The example omits those generated fields and excludes metrics for readability.

8. Proactive Remote-Message Handling

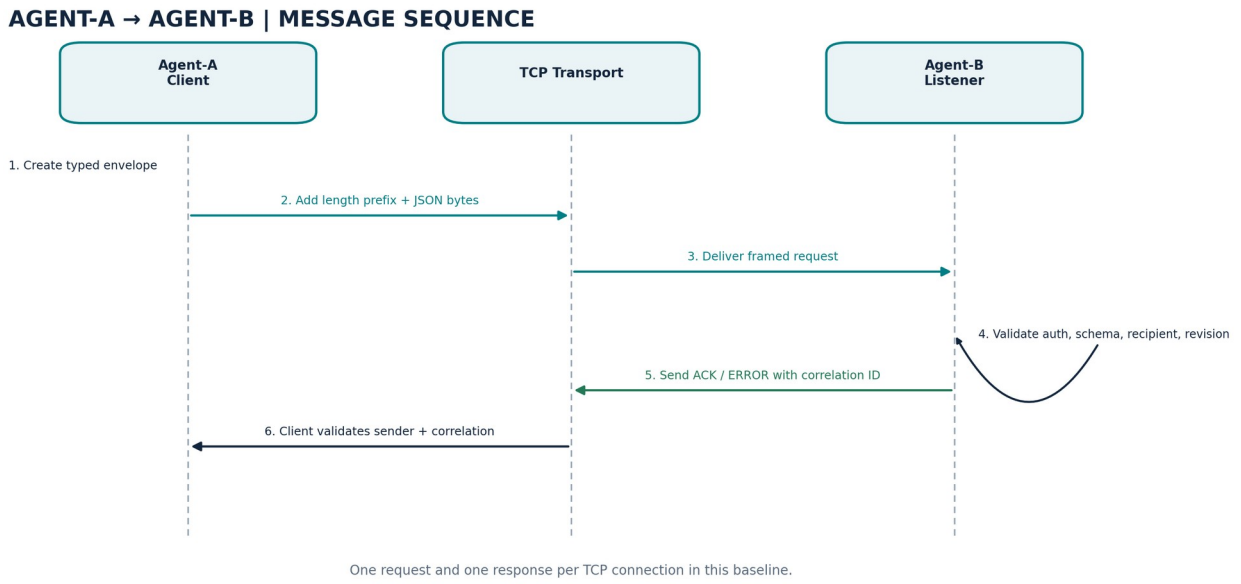


Figure 4. Message sequence: framed request, validated dispatch and correlated response.

Command	Operation	Guardrail / result
`TRIGGER_EMERGENCY_LOCKDOWN`	Set `force_block_all`; all later inspections return BLOCK	No remote unlock; ACK `lockdown_enabled`
`UPDATE_REDACTION_PATTERNS`	Validate and append custom regex patterns	Max 32 patterns, 128 chars each; reject common hazardous syntax; batch is all-or-nothing
`REQUEST_TELEMETRY_DUMP`	Return safe config and aggregate metrics	No prompt text, matched values or auth token; ACK `telemetry_ready`

8.1 Autonomous resolution

The receiver does not execute arbitrary strings as Python, shell commands or dynamic imports. It selects a handler from a fixed command vocabulary. Unknown command values return ERROR. Emergency lockdown is applied immediately and the policy engine checks it before scanning the submitted text.

8.2 Custom pattern update

Instead of inventing undocumented 2026 token formats, the implementation accepts operator-supplied patterns under length and count limits. It validates compilation and rejects common high-risk constructs such as lookarounds, backreferences and nested quantifiers. These checks are defense-in-depth, not a formal proof against regular-expression denial of service. Production policy should use signed, reviewed bundles and consider safe-regex engines or declarative token rules.

8.3 Telemetry

The telemetry snapshot includes the current safe policy state plus counts for inspections, blocks, redactions, reviews, allows and mean measured inspection latency. Counters are in memory and reset on restart. No latency figure from the supplied sample is represented as a measurement.

9. Codebase Walkthrough

Module	Responsibility
`models.py`	Typed enums, `AgentState`, message envelopes, finding/result models and timestamps
`transport.py`	Frame encoding, exact-read decoding, size limits and correlated request helper
`server.py`	Agent-B listener, optional shared-token comparison and bounded worker threads
`client.py`	Agent-A CLI for PING, state sync, inspection and proactive commands
`agent.py`	Message dispatch, state synchronization, lockdown/pattern/telemetry handlers, counters and replay window
`detector.py`	Baseline detectors, custom-pattern validation, overlap handling and redaction
`policy.py`	Risk-priority engine and pre-checks for lockdown/oversized content
`gateway.py`	Provider allow-list and preflight preparation; no external model call
`audit.py`	SQLite WAL event schema; stores metadata only
`api.py`	Optional local FastAPI routes: health, state, inspect, gateway preflight
`tests/`	Policy, gateway, socket framing and live loopback integration coverage

9.1 Implementation decisions worth noting

- An RLock serializes state transitions and counter updates across handler threads.
- Finding responses omit matched values; audit events omit raw prompts and sanitized text.
- The client verifies response sender, recipient and correlation ID.
- Resource limits are explicit: frame limit, worker count, replay-ID cache and custom-pattern cap.
- The provider preflight fails closed for unallowlisted destinations and BLOCK/REVIEW decisions.

10. Setup and Dual-Terminal Execution

10.1 Requirements

Python 3.10+ is required. The core runtime uses Pydantic 2.6+. FastAPI/Uvicorn are optional; no cloud account, cloud credentials or paid API calls are required for the socket demonstration.

10.2 Install on Windows PowerShell

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -e ".[dev,api]"
```

For the socket core, `pip install -e ".[dev]"` is enough. In an offline environment where build tools already exist, `python -m pip install -e ".[dev]" --no-build-isolation` avoids creating an isolated environment that attempts to download build tools.

10.3 Terminal 1 — launch Agent-B

```
$env:CIPHERMESH_SHARED_TOKEN="demo-change-this-token"
python -m ciphermesh.server --host 127.0.0.1 --port 9052 --agent-id AGENT_B_NODE_01 --audit-path agent_b_audit.db
```

Expected startup line:

```
[AGENT_B_NODE_01] listening on 127.0.0.1:9052; Ctrl+C to stop
```

10.4 Terminal 2 — run Agent-A commands

```
$env:CIPHERMESH_SHARED_TOKEN="demo-change-this-token"
python -m ciphermesh.client --host 127.0.0.1 --port 9052 ping
python -m ciphermesh.client --host 127.0.0.1 --port 9052 sync-state
python -m ciphermesh.client --host 127.0.0.1 --port 9052 inspect --text "Contact alice@example.com"
python -m ciphermesh.client --host 127.0.0.1 --port 9052 inspect --text "aws_access_key=AKIA1234567890ABCDEF"
python -m ciphermesh.client --host 127.0.0.1 --port 9052 update-patterns "PROJECT-CODE-[A-Z]{4}"
python -m ciphermesh.client --host 127.0.0.1 --port 9052 inspect --text "Reference PROJECT-CODE-ABCD"
python -m ciphermesh.client --host 127.0.0.1 --port 9052 telemetry
```

Run lockdown last because it deliberately blocks all later requests:

```
python -m ciphermesh.client --host 127.0.0.1 --port 9052 lockdown
python -m ciphermesh.client --host 127.0.0.1 --port 9052 inspect --text "clean text"
```

For macOS/Linux, use `export CIPHERMESH_SHARED_TOKEN="demo-change-this-token"`. Keep both processes on loopback. The sample token is a demo shared secret, not transport encryption.

11. Demonstration Cases and Expected Outcomes

Command / case	Behavior verified	Expected output
PING	Connectivity and request/response framing	ACK; `status: ok`
STATE_SYNC	Transfer active parameters and apply validation	ACK; `state_applied` or `state_unchanged`
Email inspection	PII detection and masking	REDACT; `[REDACTED:EMAIL]`
Synthetic AWS-like key	Critical secret priority	BLOCK; no sanitized text
Custom pattern update	Proactive policy change affects future input	ACK; later matching input REDACT
Telemetry request	Metadata-safe status snapshot	ACK; `telemetry_ready`
Lockdown	Remote instruction changes policy state	ACK; later clean input BLOCK
Fragmented frame test	Header/body can arrive in partial reads	Complete envelope reconstructed
Oversized/malformed frames	Input limits and schema validation	`ProtocolError`; no handler dispatch

11.1 Observed live verification

A live command-line run was completed using a separate loopback port (19052) to avoid collision with any service on the default port. Agent-B logged received PING, STATE_SYNC, INSPECT and COMMAND_INSTRUCTION messages and returned ACK responses. The client showed REDACT for the test email, BLOCK for the synthetic credential-like token, REDACT for the operator-added pattern, telemetry readiness, and BLOCK after lockdown. The shipped default remains 9052.

12. Stability and Failure Handling

Condition	Current mitigation	Remaining gap
Partial TCP delivery	`recv_exact` loops; `sendall` writes fully	One message per connection; no multiplexed streaming
Peer closes mid-frame	Connection exception; handler closes safely	No automated retry/backoff
Zero / oversize length	Reject before reading body	OS-level resource usage still exists
Malformed JSON/schema	Protocol error; no typed dispatch	Could add more structured error frame before parse
Stalled peer	Socket timeout	Timeout currently static
Worker saturation	Bounded semaphore rejects excess clients	No request queue/backpressure protocol
Concurrent state changes	RLock protects process-local state	No cross-replica coordination
Replay ID	Bounded 4,096-ID in-memory window	Resets on restart; not cryptographic
Stale/same-revision update	Stale rejected; same identical no-ops; same changed rejected	Policy is not digitally signed
Custom regex hazard	Count/length cap and common-construct checks	Heuristic, not a formal ReDoS guarantee
Wrong/missing token	Configured token required; constant-time compare	Plain TCP does not encrypt the token

13. Security and Privacy Analysis

13.1 Trust boundaries

Treat application text, message payloads and custom patterns as untrusted input. The listener is a trust boundary: it validates frame length and the typed envelope before dispatch. Schema validity is not authorization, so the local demo token and loopback binding are recommended. Production control should be per-agent and per-command.

13.2 Threats considered

- Accidental credential / PII disclosure through LLM prompts.
- Oversized messages and slow peers attempting to consume resources.
- Truncated frames, malformed JSON and mismatched response envelopes.
- State tampering, stale policies and duplicate messages.
- Unauthorized remote commands and sensitive telemetry leakage.
- Unsafe custom regex updates.

13.3 Controls implemented

- Inspection preflight returns no provider content for BLOCK or REVIEW, and returns sanitized text for REDACT.

- Finding JSON and audit records omit matched values and raw prompt text.
- Frames are size-limited, schema-validated and read fully before dispatch.
- State sync uses a field allow-list, revision checks and validation before assignment.
- Remote commands are fixed enum-like actions, not arbitrary code.
- A shared-token comparison uses `hmac.compare_digest` when configured.
- Telemetry contains only approved configuration and aggregate counts.

13.4 Required before production

Add TLS/mTLS with certificate validation, certificate rotation, per-agent authorization, signed policy bundles, persistent anti-replay controls, rate limiting, structured log retention and a human-review workflow. Run static analysis and adversarial detector tests. Twenty-two tests indicate functional behavior, not a security certification.

14. Verification and Evaluation Plan

14.1 Automated test result

TEST RESULT: 22 PASSED — verified in the local environment with `python -m pytest -q`.

Coverage includes ALLOW/REDACT/REVIEW/BLOCK/oversize behavior, no-value finding serialization, state revision rules, lockdown, pattern guardrails, telemetry scrubbing, wrong recipient, frame round trip, fragmented delivery, oversized and invalid frames, provider preflight enforcement, and a live loopback TCP state-sync/inspection sequence.

14.2 Metrics for a research-quality evaluation

Metric	Measurement	Why it matters
Precision	TP / (TP + FP) on labelled cases	False-positive and usability impact
Recall	TP / (TP + FN)	Missed-disclosure risk
False-block rate	Benign requests blocked / benign requests	User friction
P50 / P95 latency	Repeated `perf_counter` observations	Typical and tail request overhead
TCP success rate	ACK responses / valid requests	Endpoint reliability
Schema rejection correctness	Malformed frames correctly rejected / malformed inputs	Protocol safety
State-sync correctness	Expected revision applied without forbidden mutation	Control-plane integrity
Audit leakage	Check stored events for raw content or matched values	Privacy of observability

No precision/recall, throughput or latency benchmark figures are claimed because a labelled corpus and repeatable load benchmark have not yet been run. The telemetry mean is a process-local running average, not an evaluation result.

15. Limitations and Next Milestones

Milestone 1 — Harden the demonstration

- Add structured logs, rate limiting and graceful shutdown.
- Persist policy revision and replay state if restart continuity is required.

- Add property-based/fuzz tests for JSON, frame boundaries and Unicode edge cases.
- Replace free-form remote regex with a smaller declarative pattern language or safe-regex engine.

Milestone 2 — Complete the privacy gateway

- Integrate Presidio behind a detector interface and verify model dependency footprint.
- Create a synthetic labelled corpus with benign, sensitive and obfuscated cases.
- Add a human review queue with reasons, approve/reject and expiry.
- Implement provider adapters that enforce the preflight result immediately before each API call.

Milestone 3 — Network productionization

- Enable TLS/mTLS, individual agent identities and per-command authorization.
- Sign and version policy bundles; add durable anti-replay and idempotency semantics.
- Add health metrics, tracing, supervised deployment and deployment manifests.

Milestone 4 — Multi-cloud evaluation

- Verify real provider request/response paths for AWS Bedrock, Azure AI and Google Vertex AI.
- Measure utility/security trade-offs by destination and content type.
- Compare regex baseline against Presidio-enhanced detection on a held-out labelled set.

16. Assignment Deliverable Mapping

Assignment requirement	Evidence in this submission	Status
Client-server / P2P sockets or RPC	Agent-A CLI and Agent-B TCP listener	Implemented client-server baseline
Serialized typed message interface	Pydantic `WireMessage`, UTF-8 JSON, length-prefixed frame	Implemented
Transfer agent active state	`sync-state` carries revision and policy parameters	Implemented
Parse proactive messages	`COMMAND_INSTRUCTION` dispatcher	Implemented
Respond autonomously	Lockdown, custom-pattern update, telemetry snapshot	Implemented
Network stability	Exact-read, sendall, max frame, timeouts, bounded workers	Implemented baseline
Data integrity	Schema validation, allow-list, revision checks and correlation	Implemented baseline
Dual-terminal demo	Agent-B listener and Agent-A client commands	Documented and live-tested
Complete professional documentation	Architecture, PEAS, protocol, runbook, tests, threat/limitation analysis	Included
Live cloud providers	Provider preflight only; no real cloud calls	Future work; not claimed complete

17. Conclusion

CipherMesh demonstrates both the privacy decision engine and a networked autonomous-agent control path. The key achievement is not simply sending JSON over a socket: Agent-B validates the frame and schema, checks recipient and message identity, applies only allowed state changes, executes a fixed instruction vocabulary and acknowledges the result. The design is reproducible on one machine and the current test suite covers core behavior and an actual loopback TCP sequence.

The recommended next step is to preserve this constrained wire contract while hardening transport identity, encryption and policy signing; then connect the privacy preflight to real provider adapters. This creates a responsible path from educational baseline to a more complete multi-cloud gateway without overstating current capabilities.

References

[1] OWASP Foundation. “Top 10 for Large Language Model Applications.” Current project page, accessed 12 October 2026. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[2] Python Software Foundation. “socket — Low-level networking interface.” Python documentation, accessed 12 October 2026. https://docs.python.org/3/library/socket.html

[3] Pydantic. “Models and validation.” Pydantic documentation, accessed 12 October 2026. https://pydantic.dev/docs/validation/latest/get-started/

[4] Microsoft Presidio. “Text anonymization / Analyzer / Anonymizer.” Official documentation, accessed 12 October 2026. https://microsoft.github.io/presidio/text_anonymization/

Appendix A — CLI Quick Reference

```
# Terminal 1 — Agent-B
$env:CIPHERMESH_SHARED_TOKEN="demo-change-this-token"
python -m ciphermesh.server --host 127.0.0.1 --port 9052 --agent-id AGENT_B_NODE_01

# Terminal 2 — Agent-A
$env:CIPHERMESH_SHARED_TOKEN="demo-change-this-token"
python -m ciphermesh.client --port 9052 ping
python -m ciphermesh.client --port 9052 sync-state
python -m ciphermesh.client --port 9052 inspect --text "Contact alice@example.com"
python -m ciphermesh.client --port 9052 update-patterns "PROJECT-CODE-[A-Z]{4}"
python -m ciphermesh.client --port 9052 telemetry
python -m ciphermesh.client --port 9052 lockdown
```

Appendix B — Preflight Result Semantics

Decision	Meaning	Provider-ready text
ALLOW	No configured trigger found	Original text
REDACT	PII/custom pattern replaced	Sanitized text only
REVIEW	Human review required	`null`
BLOCK	Critical secret, lockdown, oversize or unapproved provider	`null`

Appendix C — Repository Inventory & Submission Checklist

```
ciphermesh/
├── ciphermesh/      models, detector, policy, agent, transport, server, client, gateway, audit, API
├── tests/           policy, transport, network integration and gateway tests
├── docs/            implementation report and architecture illustrations
├── README.md        install, runbook, configuration and limitations
├── pyproject.toml   package metadata and optional dependency groups
└── requirements.txt
```

C.1 Reproducibility acceptance checklist

Check	Pass condition
Environment	Python 3.10+ and virtual environment activated
Receiver	Agent-B logs the configured host and port; no bind error
Authentication	Client and server use the same demo token when enabled
State transfer	STATE_SYNC returns ACK and the expected policy revision
Autonomous actions	Email REDACTs, synthetic critical key BLOCKs, pattern update affects later inspection
Privacy / shutdown	Telemetry excludes raw prompt values; lockdown blocks later content; Ctrl+C stops server

C.2 Quick troubleshooting

Symptom	Likely cause / action
Connection refused	Start Agent-B first; verify host and port match on both terminals.
Authentication failed	Use the same `CIPHERMESH_SHARED_TOKEN` in both terminals.
Address already in use	Choose a free port for `--port` and pass it to Agent-A too.
STATE_SYNC rejected	Restart Agent-B for a clean demo or inspect the revision and policy fields.
Unexpected BLOCK after lockdown	Expected behavior: lockdown is deliberately fail-closed; restart Agent-B to reset the in-memory demo state.